



ASSIGNMENT TITLE :

General Report on compilation of C program week (Lab 2)

Module Code: ST5003CMD

Module Name: Operating Systems, Security, and Networks

Intake: March 2025

Submission date: September 20th, 2026

Submitted By

Name : Ujwal Acharya
Student ID : 16537235
College ID : 250036

Submitted To

Rupak Rajbanshi
Lecturer

Softwarica College of IT & E-Commerce
In Collaboration with Coventry University
Kathmandu, Nepal

Table of Contents

Table of Figures	3
1. Structure of a .c file	4
2. GCC installation	6
3. C compilation	7
4. Questions	10
5. Lab Tasks	13
6. GitHub repo link	19

Table of Figures

Figure 1:Code for Hello World!	4
Figure 2: Output for Hello, world!	4
Figure 3: Code for User input	4
Figure 4: Output for Input taken	4
Figure 5: Code for formatted Output	5
Figure 6:Output for formatted code	5
Figure 7:Gcc installation	6
Figure 8:Gcc verification	6
Figure 9: Preprocessing code	7
Figure 10: Compiling code	8
Figure 11: Assembling code	8
Figure 12: Linking code and executing file	9
Figure 13: Purpose of #include directive	10
Figure 14: Output of Hello, World!	10
Figure 15: Asks the user to input the value and displays it	11
Figure 16: Output of displaying user input	11
Figure 17: Creating hello.c file	13
Figure 18: Editing hello.c file using nano	13
Figure 19: Output and verifying exit status using echo \$?	13
Figure 20: Creating hello.c	14
Figure 21: Editing hello.c using nano	14
Figure 22: Output and verifying exit using echo \$?	14
Figure 23: Creating input.c	15
Figure 24: Editing input.c to take input and display it	15
Figure 25: Displaying output taken from the user	15
Figure 26: Preprocessing	16
Figure 27: Compilation	16
Figure 28: Assembling	17
Figure 29: Linking	17
Figure 30: Creating test.c	18
Figure 31: Editing Test.c to return 50	18
Figure 32: Output and verifying exit returning 50 using echo \$?	18

1. Structure of a .c file

- i) Begins with file content enclosed by a comment, i.e /*...*/
- ii) We use #include <stdio.h>, stdio.h is c libraries
- iii) We define the main function, i.e int main(){}

a) Basic Hello world Program :

```
#include <stdio.h>

int main(){
    printf("Hello,world!\n");
    return 0;
}
```

Figure 1:Code for Hello World!

```
ujwal0608@Ubuntu:~/Desktop$ nano hello.c
ujwal0608@Ubuntu:~/Desktop$ gcc hello.c -o hello
ujwal0608@Ubuntu:~/Desktop$ ./hello
Hello, World!
```

Figure 2: Output for Hello, world!

b) Program with user input:

```
GNU nano 8.7.1
#include <stdio.h>

int main(){
    int number;
    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 0;
}
```

Figure 3: Code for User input

```
ujwal0608@Ubuntu:~/Desktop$ nano hello.c
ujwal0608@Ubuntu:~/Desktop$ gcc hello.c -o hello
ujwal0608@Ubuntu:~/Desktop$ ./hello
Enter a number: 50
You entered: 50
```

Figure 4: Output for Input taken

c) Program with formatted Output:

```
GNU nano 8.7.1 hello.c *
#include <stdio.h>

int main(){
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m): ");
    scanf("%f", &height);

    printf("\n--- User Info ---\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m\n", height);

    return 0;
}
```

Figure 5: Code for formatted Output

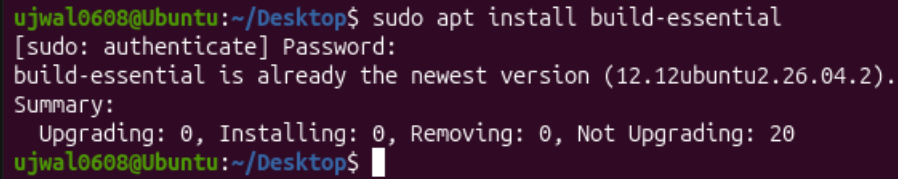
```
ujwal0608@Ubuntu:~/Desktop$ nano hello.c
ujwal0608@Ubuntu:~/Desktop$ gcc hello.c -o hello
ujwal0608@Ubuntu:~/Desktop$ ./hello
Enter your name: Ujwal
Enter your age: 23
Enter your height (m): 1.63

--- User Info ---
Name: Ujwal
Age: 23 years
Height: 1.63 m
ujwal0608@Ubuntu:~/Desktop$
```

Figure 6: Output for formatted code

2. GCC installation

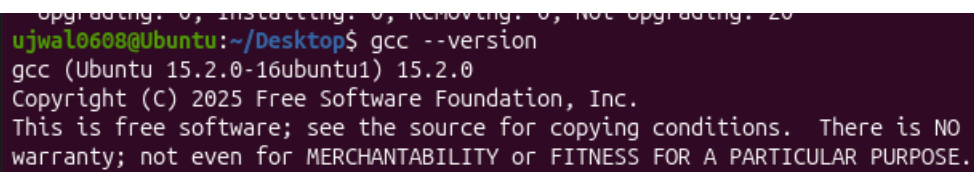
- i) To install on Linux, we use: `sudo apt install build-essential`

A terminal window with a dark purple background. The prompt is 'ujwal0608@Ubuntu:~/Desktop\$'. The command 'sudo apt install build-essential' is entered. The output shows a password prompt, a message that build-essential is already the newest version (12.12ubuntu2.26.04.2), a summary of the installation, and the command prompt again.

```
ujwal0608@Ubuntu:~/Desktop$ sudo apt install build-essential
[sudo: authenticate] Password:
build-essential is already the newest version (12.12ubuntu2.26.04.2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 20
ujwal0608@Ubuntu:~/Desktop$
```

Figure 7:Gcc installation

- ii) To check: `gcc --version`

A terminal window with a dark purple background. The prompt is 'ujwal0608@Ubuntu:~/Desktop\$'. The command 'gcc --version' is entered. The output shows the gcc version (15.2.0-16ubuntu1), copyright information, and a disclaimer.

```
ujwal0608@Ubuntu:~/Desktop$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

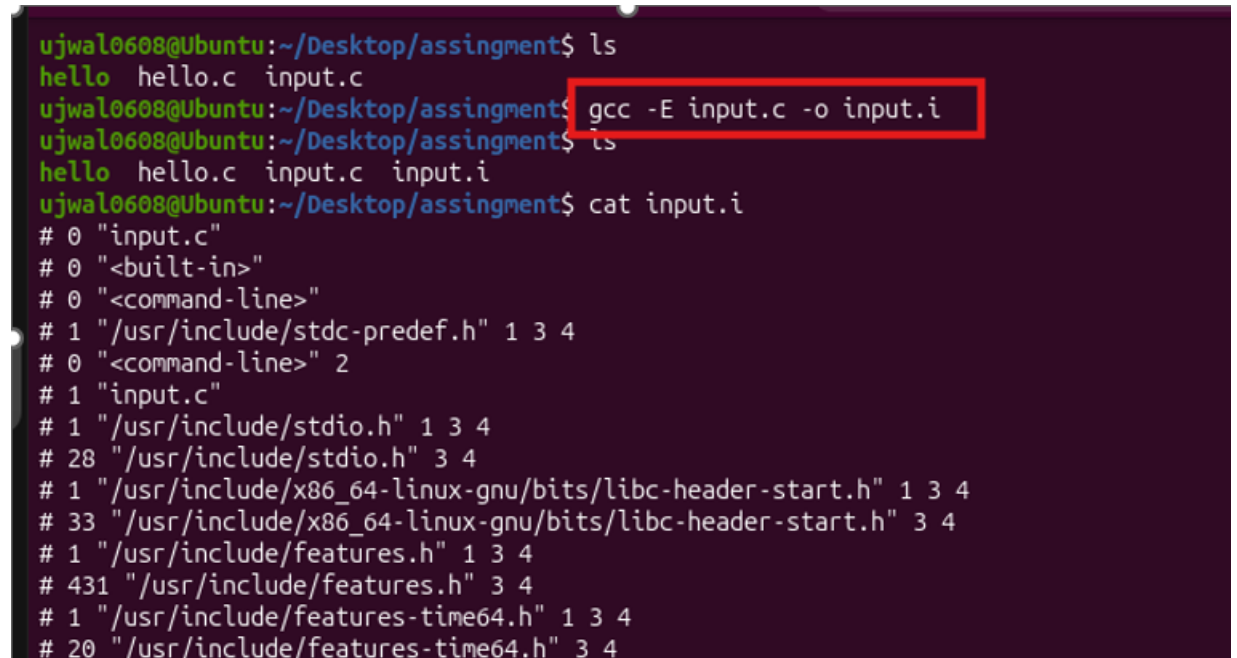
Figure 8:Gcc verification

3. C compilation

i) Stage 1:Preprocessing

At first, we have C source code; the preprocessor handles things that begin with #, i.e #include; it also removes comments. The output is also referred to as .i file. The file is called preprocessed C code.

Code : gcc -E hello.c -o hello.i

A terminal window with a dark purple background and light green text. The user is in the directory ~/Desktop/assingment. The first command is 'ls', showing 'hello.c' and 'input.c'. The second command is 'gcc -E input.c -o input.i', which is highlighted with a red rectangular box. The third command is 'ls', showing 'hello.c', 'input.c', and 'input.i'. The fourth command is 'cat input.i', displaying the preprocessed code for 'input.c', which includes system headers like stdc-predef.h, stdio.h, and various GNU C library headers.

```
ujwal0608@Ubuntu:~/Desktop/assingment$ ls
hello.c  input.c
ujwal0608@Ubuntu:~/Desktop/assingment$ gcc -E input.c -o input.i
ujwal0608@Ubuntu:~/Desktop/assingment$ ls
hello.c  input.c  input.i
ujwal0608@Ubuntu:~/Desktop/assingment$ cat input.i
# 0 "input.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "input.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 431 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
```

Figure 9: Preprocessing code

ii) Stage 2: Compiling

The converted preprocessed code is translated into assembly code (.s file).

Assembly code is used for debugging the large program.

Code :gcc -S hello.i -o hello.s

```
ujwal0608@Ubuntu:~/Desktop/assingment$ ls
hello hello.c input.c input.i
ujwal0608@Ubuntu:~/Desktop/assingment$ gcc -S input.i -o input.s
ujwal0608@Ubuntu:~/Desktop/assingment$ ls
hello hello.c input.c input.i input.s
ujwal0608@Ubuntu:~/Desktop/assingment$ head input.s
.file "input.c"
.text
.section .rodata
.LC0:
.string "Enter a number: "
.LC1:
.string "%d"
.LC2:
.string "You entered: %d\n"
.text
ujwal0608@Ubuntu:~/Desktop/assingment$
```

Figure 10: Compiling code

iii) Stage 3:Assembling

The converted assembly code (.s file) is converted into machine code (object file, i.e .o file) using an assembler.

Code: gcc -c hello.s -o hello.o

```
tujwal0608@Ubuntu:~/Desktop/assignment$ gcc -c input.s -o input.o
tujwal0608@Ubuntu:~/Desktop/assignment$ ls
hello hello.c input.c input.i input.o input.s
tujwal0608@Ubuntu:~/Desktop/assignment$ head input.o
UUHHHooHooH%(HoEe1oHoHoNoHoEohHoHo[?]oEoHoooHo[?]oHoUodH+%(toEnter a number: %dYou entered: %d
GCC: (Ubuntu 15.2.0-16ubuntu1) 15.2.0GNUzRX
u                                         ~EoC
FooooooooP_oooooooXooooooooo .symtab.strtab.shstrtab.rela.text.data.bss.rodata.comment.note.GNU-stack.note.gnu
u.property.rela.eh_frame @~♦
                               &ooo1o%9O♦'B

Rj0eq
ho
(5 tujwal0608@Ubuntu:~/Desktop/assignment$
```

Figure 11: Assembling code

iv) **Stage 4:Linking**

It combines object files and library files and converts them into an executable file (.exe) using a linker.

Code: gcc hello.o -o hello

To run: ./hello

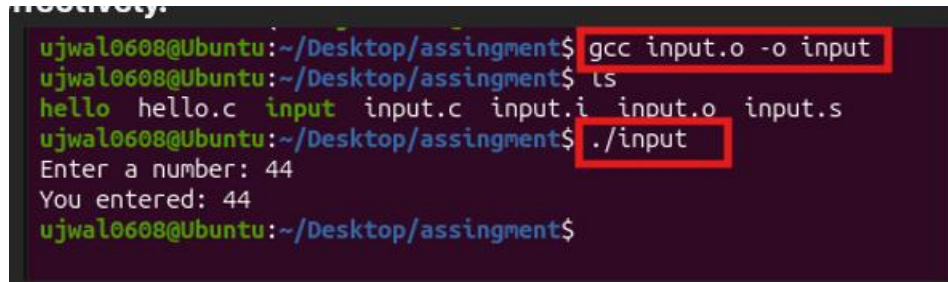
A terminal window with a dark background and light green text. The prompt is 'ujwal0608@Ubuntu:~/Desktop/assingment\$'. The first command is 'gcc input.o -o input', which is highlighted with a red box. The second command is 'ls', showing the output 'hello hello.c input input.c input.i input.o input.s'. The third command is './input', also highlighted with a red box. The program then prompts 'Enter a number: 44' and 'You entered: 44'. The final prompt is 'ujwal0608@Ubuntu:~/Desktop/assingment\$'.

Figure 12: Linking code and executing file

4. Questions

a) What is the purpose of the #include directive in C programming?

Provide an example of its usage.

The **#include** directive in C includes the contents of a header file in a program before compilation. Header files contain many useful functions and declarations.

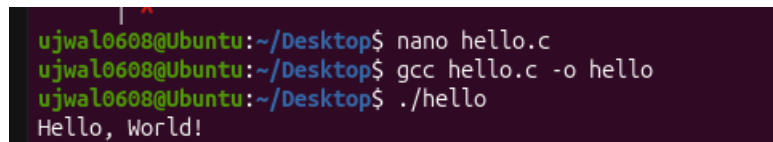


```
#include <stdio.h>

int main(){
    printf("Hello,world!\n");
    return 0;
}
```

Figure 13: Purpose of #include directive

Output:



```
ujwal0608@Ubuntu:~/Desktop$ nano hello.c
ujwal0608@Ubuntu:~/Desktop$ gcc hello.c -o hello
ujwal0608@Ubuntu:~/Desktop$ ./hello
Hello, World!
```

Figure 14: Output of Hello, World!

In this, **#include <stdio.h>** provides the standard input/output library, which provides functions such as `printf()`.

b) Explain the difference between the four stages of compilation.

What file extension is produced at each stage?

i) Preprocessing

It handles `#include`, `#define`, and removes comments.

`.i` file extension is produced after preprocessing.

ii) Compiling :

It converts the preprocessed C code into assembly code.

`.s` extension is produced after Compiling.

iii) Assembling:

It converts assembly code into machine/object code.

`.o` extension file is produced after assembling

iv) Linking:

It combines object code with necessary libraries to create the final executable file.

On Windows, **.exe** extension, but in linux **no extension**.

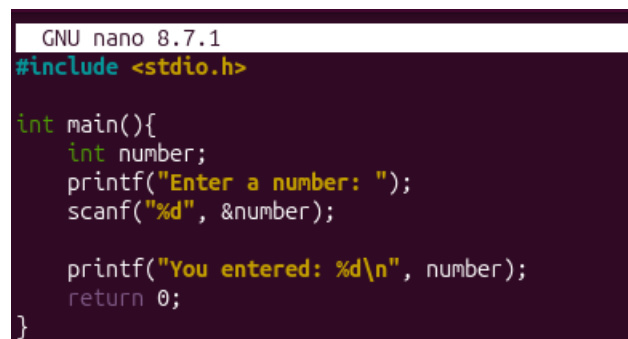
c) What does a return value of 0 from the main() function indicate to the operating system? What about a non-zero return value?

A return value of 0 from main() indicates that the program executed successfully without errors. While Non-zero return value indicates that the program ended with an error or abnormal conditions.

i.e., 0 -> Success and Non-zero -> Error

d) Write a C program that :

- **Declare an integer variable**
- **Asks the user to input a value**
- **Displays the entered value back to the user**

A screenshot of a terminal window showing a C program being edited in the nano 8.7.1 editor. The code is as follows:

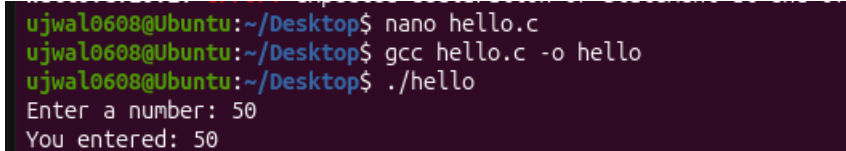
```
GNU nano 8.7.1
#include <stdio.h>

int main(){
    int number;
    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 0;
}
```

Figure 15: Asks the user to input the value and displays it

Output :

A screenshot of a terminal window showing the execution of the C program. The user enters the number 50, and the program outputs "You entered: 50".

```
ujwal0608@Ubuntu:~/Desktop$ nano hello.c
ujwal0608@Ubuntu:~/Desktop$ gcc hello.c -o hello
ujwal0608@Ubuntu:~/Desktop$ ./hello
Enter a number: 50
You entered: 50
```

Figure 16: Output of displaying user input

e) What GCC command would you use to :

- **Only preprocess a file without compiling?**
- **Generate assembly code but not assemble it?**
- **Create an object file without linking?**

1. To preprocess a C file only :

gcc -E program.c -o program.i

2. To generate assembly code but not assemble it :

gcc -S program.c -o program.s

3. Creating an object file without linking :

gcc -c program.c -o program.o

f) Explain why C is called a “compiled language” and how this differs from interpreted languages.

C is called a compiled language because a C program is first translated(compiled) into machine code by a compiler before it is executed. While in an interpreted language, the program is generally executed by an interpreter, in which source code is translated and executed during runtime .it usually doesnot produce standalone executable in the same way as a compiled language.

5. Lab Tasks

a) Task 1:

Compiled hello.c with return 0 and verified exit status using echo \$?

```
ujwal0608@Ubuntu:~/Desktop$ mkdir assingment
ujwal0608@Ubuntu:~/Desktop$ ls
assingment  c_folder  my_os_folder  newfile.txt  test  test.c
ujwal0608@Ubuntu:~/Desktop$ cd assingment
ujwal0608@Ubuntu:~/Desktop/assingment$ ls
ujwal0608@Ubuntu:~/Desktop/assingment$ nano hello.c
ujwal0608@Ubuntu:~/Desktop/assingment$
```

Figure 17: Creating hello.c file

```
GNU nano 8.7.1 hello.c *
#include <stdio.h>

int main() {
    printf("Hello World\n");
    return 0;
}
```

Figure 18: Editing hello.c file using nano

```
ujwal0608@Ubuntu:~/Desktop/assingment$ gcc hello.c -o hello
ujwal0608@Ubuntu:~/Desktop/assingment$ ./hello
Hello World
ujwal0608@Ubuntu:~/Desktop/assingment$ echo $?
0
ujwal0608@Ubuntu:~/Desktop/assingment$
```

Figure 19: Output and verifying exit status using echo \$?

Explanation:

Return 0 tells Os that the program completed successfully, and echo \$? Displays the exit status of the most recently executed command.

b) Task 2:

Modified the program to return 1 and observed a different exit status.

```
ujwal0608@Ubuntu:~/Desktop/assingment$ nano hello.c
ujwal0608@Ubuntu:~/Desktop/assingment$
```

Figure 20: Creating hello.c



```
GNU nano 8.7.1 hello.c *
#include <stdio.h>

int main() {
    printf("Hello World\n");
    return 1;
}
```

Figure 21: Editing hello.c using nano

```
ujwal0608@Ubuntu:~/Desktop/assingment$ gcc hello.c -o hello
ujwal0608@Ubuntu:~/Desktop/assingment$ ./hello
Hello World
ujwal0608@Ubuntu:~/Desktop/assingment$ echo $?
1
ujwal0608@Ubuntu:~/Desktop/assingment$
```

Figure 22: Output and verifying exit using echo \$?

Explanation:

The program was modified to return 1. After compiling and running the program, echo \$? Displayed 1, confirming the program returned a non -zero exit status.

c) **Task 3:**

Created a program accepting user input and displaying it

```
ujwal0608@Ubuntu:~/Desktop/assingment$ nano input.c
ujwal0608@Ubuntu:~/Desktop/assingment$
```

Figure 23: Creating input.c

```
GNU nano 8.7.1
# include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 0;
}
```

Figure 24: Editing input.c to take input and display it

```
ujwal0608@Ubuntu:~/Desktop/assingment$ gcc input.c -o input
ujwal0608@Ubuntu:~/Desktop/assingment$ ./input
Enter a number: 55
You entered: 55
ujwal0608@Ubuntu:~/Desktop/assingment$
```

Figure 25: Displaying output taken from the user

Explanation :

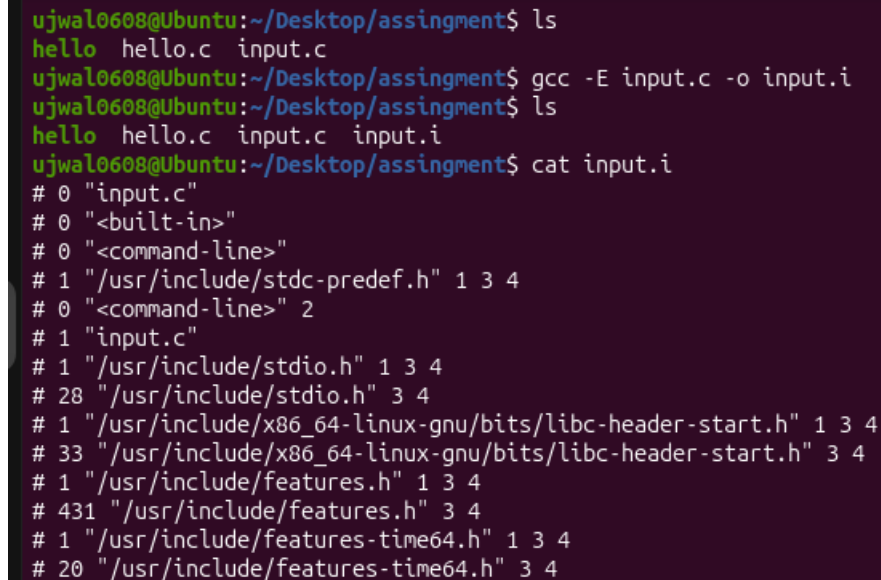
A C program was created to accept an integer value from the user using `scanf()` and display the entered value using `printf()`.

d) Task 4:

Executed all four compilation stages separately and documented output files

Stage 1: Preprocessing

It converts a .c file into .i file. It also handles #include and removes comments.

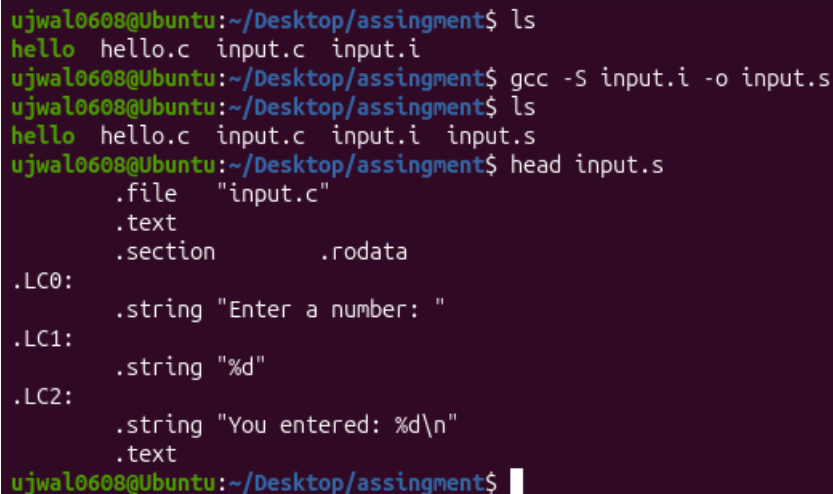


```
ujwal0608@Ubuntu:~/Desktop/assingment$ ls
hello hello.c input.c
ujwal0608@Ubuntu:~/Desktop/assingment$ gcc -E input.c -o input.i
ujwal0608@Ubuntu:~/Desktop/assingment$ ls
hello hello.c input.c input.i
ujwal0608@Ubuntu:~/Desktop/assingment$ cat input.i
# 0 "input.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "input.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 431 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
```

Figure 26: Preprocessing

Stage 2: Compilation

It convert .i file into .s file. It converts preprocessed code into assembly code.



```
ujwal0608@Ubuntu:~/Desktop/assingment$ ls
hello hello.c input.c input.i
ujwal0608@Ubuntu:~/Desktop/assingment$ gcc -S input.i -o input.s
ujwal0608@Ubuntu:~/Desktop/assingment$ ls
hello hello.c input.c input.i input.s
ujwal0608@Ubuntu:~/Desktop/assingment$ head input.s
.file "input.c"
.text
.section .rodata
.LC0:
.string "Enter a number: "
.LC1:
.string "%d"
.LC2:
.string "You entered: %d\n"
.text
ujwal0608@Ubuntu:~/Desktop/assingment$
```

Figure 27: Compilation

Stage 3:Assembling

It converts .s file into .o file. It converts assembly code into machine code or object code.

```

ujwal0608@Ubuntu:~/Desktop/assingment$ gcc -c input.s -o input.o
ujwal0608@Ubuntu:~/Desktop/assingment$ ls
hello hello.c input.c input.i input.o input.s
ujwal0608@Ubuntu:~/Desktop/assingment$ head input.o
00000000: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
GCC: (Ubuntu 15.2.0-16ubuntu1) 15.2.0 GNUzRx
u
~E0C
F0000000P_00000000X00000000 .symtab.strtab.shstrtab.rela.text.data.bss.rodata.comment.note.GNU-stack.note.gnu
u.property.rela.eh_frame @~`0
&0010%900'B
Rj0e@
h0
(5 tujwal0608@Ubuntu:~/Desktop/assingment$

```

Figure 28: Assembling

Stage 4: Linking

It converts object code into an executable file and also links necessary libraries to work effectively.

```
ujwal0608@Ubuntu:~/Desktop/assingment$ gcc input.o -o input
ujwal0608@Ubuntu:~/Desktop/assingment$ ls
hello hello.c input input.c input.i input.o input.s
ujwal0608@Ubuntu:~/Desktop/assingment$ ./input
Enter a number: 44
You entered: 44
ujwal0608@Ubuntu:~/Desktop/assingment$
```

Figure 29: Linking

e) Task 5:

Compiled and ran the program, returning the integer value 50

```
ujwal0608@Ubuntu:~/Desktop/assingment$ ls
hello hello.c input input.c input.i input.o input.s
ujwal0608@Ubuntu:~/Desktop/assingment$ nano test.c
ujwal0608@Ubuntu:~/Desktop/assingment$
```

Figure 30: Creating test.c

```
GNU nano 8.7.1
#include <stdio.h>

int main(){
    printf("Hello World\n");
    return 50;
}
```

Figure 31: Editing Test.c to return 50

```
ujwal0608@Ubuntu:~/Desktop/assingment$ ls
hello hello.c input input.c input.i input.o input.s
ujwal0608@Ubuntu:~/Desktop/assingment$ nano test.c
ujwal0608@Ubuntu:~/Desktop/assingment$ gcc test.c -o test
ujwal0608@Ubuntu:~/Desktop/assingment$ ./test
Hello World
ujwal0608@Ubuntu:~/Desktop/assingment$ echo $?
50
ujwal0608@Ubuntu:~/Desktop/assingment$
```

Figure 32: Output and verifying exit returning 50 using echo \$?

Explanation:

The program was modified to return 50. After compiling and running the program, echo \$? Displayed 50, confirming the program returned a non -zero exit status.

6. GitHub repo link

<https://github.com/ujwal-2003/Operating-Systems-Security-and-Networks.git>